ROADMAP v2.0 · خريطة الطريق

متجر إلكتروني متكامل

دمج مشروع الـ `ASP.NET Core API` ومشروع الـ `React` في مشروع `Full Stack` واحد، بقاعدة بيانات واضحة، ومنشور على الإنترنت، بنفس طريقة مشروع الجيم

`.NET 10` `ASP.NET Core Web API` `EF Core 10` `SQL Server` `Identity + JWT` `Stripe (test)` `React 19` `TypeScript` `Vite` `React Router 7` `xUnit` `Playwright`

٣٢	٢٣	١٦	٣
مشكلة لقيناها في المشروعين	مشكلة اتحلت في B0 → B2	مرحلة (٨ باك + ٧ فرونت + نشر)	أدوار: زائر · عميل · مدير

المشروع	`F:\Full Stack Projects\Project_5_Ecommerce` · فرع `fullstack` · آخر دمج `9013361`
المصدر الأصلي	`github.com/Fady519/E_CommerceSolution` (الـ API) و `github.com/Fady519/Ecommerce-Website` (الرياكت)، اتجمعوا في ريبو واحد
طريقة الفحص	قراءة كل ملفات الكود في المشروعين، تشغيل الـ `API` على قاعدة حقيقية، واختبار كل رابط بسكريبت
بيئة الجهاز	`Windows · .NET SDK 10 · SQL Server (Server=.) · Node 22` · مفيش `Redis`
المرجع	مشروع الجيم `Project_4_GymManagementSystem` (نفس الطريقة: مراحل B ثم F ثم D)
المالك	فادي قيصر · `Fady519`
آخر تحديث	١٠ أكتوبر ٢٠٢٦ · بعد انتهاء المرحلة B2

الفهرس

١ الملخص في صفحة واحدة
٢ المشروع الحالي: الحاجات الكويسة اللي هنبني عليها
٣ كل المشاكل اللي لقيناها
٤ القرارات المعمارية والتقنيات
٥ شكل البيانات (الجداول)
٦ قواعد العمل لكل جزء
٧ الأدوار والصلاحيات
٨ قايمة الـ API Endpoints
٩ المميزات حسب الأولوية
١٠ الجدول الزمني وترتيب المراحل
١١ مراحل الباك إند B0 → B7
١٢ معمارية الفرونت إند
١٣ مراحل الفرونت إند F0 → F6
١٤ النشر والتوثيق D1
١٥ طريقة الشغل و Git
١٦ إزاي يتكتب في السيرة الذاتية
١٧ قرارات محتاجة رأيك
١٨ قاموس المصطلحات

١ الملخص في صفحة واحدة

✅ اللي خلص

التجميع + B2 + B1 + B0

المشروعين بقوا في ريبو واحد. الباك إند اترقى لـ `.NET 10` واتنضف، وبقى ليه قاعدة بيانات واحدة فيها منتجاتك و٣٠ عميل و٨٠ طلب تجريبي، وبيشتغل على جهازك من غير أعطال. وفي B2 اتقفلت ثغرة سرقة الحسابات، وبقى فيه أدوار وصلاحيات، وحماية لتسجيل الدخول.

⏳ المرحلة الحالية

B3 · المنتجات وإدارتها

المدير يضيف ويعدل ويخفي منتجات بصورها ومخزونها، وبحث وفلترة أحسن للزائر، والكاش يتمسح مع أي تعديل.

B0	B1	B2	B3	B4	B5	B6	B7	F0	F1	F2	F3
الأساس	الداتا بيز	الحسابات	المنتجات	الطلبات	الدفع	الإحصائيات	الاختبارات	التجهيز	الدخول	المتجر	الشراء

F4	F5	F6	D1
المدير	التشطيب	الجودة	النشر

خلصت · الحالية · لسه

الفكرة في ٣ سطور

1. **الباك إند الأول:** نصلّح ونكمّل الـ `API` لحد ما يبقى سليم وآمن ومتختبر (B0 → B7).
2. **الفرونت بعده:** نحول الرياكت لـ `TypeScript`، نشيل البيانات الوهمية، ونوصله بالـ `API` الحقيقي، ونزود الشراء ولوحة المدير (F0 → F6).
3. **النشر في الآخر:** الـ `API` على `MonsterASP` والفرونت على `Vercel`، وريدمي بالصور، ولينك تحطه في السيرة الذاتية ولينكد إن (D1).

هيبقى شكله إيه في الآخر؟

🛍️ الزائر

يتصفح المنتجات، يبحث ويفلتر بالماركة والنوع والسعر، ويحط في السلة من غير ما يسجل.

🥷 العميل

يسجل، يشتري بكارت تجريبي أو كاش عند الاستلام، ويتابع طلباته وحالتها، ويعدل عنوانه.

🛠️ المدير

لوحة تحكم: المنتجات بالصور والمخزون، الطلبات وتغيير حالتها، وإحصائيات المبيعات.

٢ المشروع الحالي: الحاجات الكويسة اللي هنبني عليها

قبل ما نتكلم عن المشاكل: المشروعين فيهم أساس كويس، ومش هنرميه. ده اللي هنحافظ عليه ونبني فوقه:

💻 الباك إند

- **تقسيم طبقات سليم** (`Onion Architecture`): `Domain` و `Persistence` و `Services` و `Presentation` و `Web` كل واحد في مشروع لوحده.
- **نمط الـ** `Specification` للاستعلامات، و `Generic Repository` و `Unit of Work` .
- **نمط الـ** `Result` بدل رمي الأخطاء في كل حتة، ورد موحد بالـ `ProblemDetails` .
- المنتجات فيها ترقيم صفحات وفلترة بالماركة والنوع وترتيب وبحث من الأول.
- الطلب بياخد نسخة من بيانات المنتج والعنوان (`snapshot`)، وده تصميم صح.
- تكامل مبدئي مع `Stripe` (`Payment Intent`).

🎨 الفرونت إند

- تصميم حلو ومتجاوب مع الموبايل، وهوية واضحة.
- سلايدرات `Swiper` وحركات `Framer Motion` وانتقالات بين الصفحات.
- صفحات جاهزة الشكل: الرئيسية، تفاصيل المنتج، السلة، المفضلة، البحث، التصنيفات، المدونة، من نحن، تواصل معنا.
- نماذج بـ `react-hook-form` ، وتنبيهات بـ `react-hot-toast` .
- أحدث نسخ: `React 19` و `Vite 7` و `React Router 7` .

هنحتفظ بـ: التصميم، ملفات الـ `CSS` ، السلايدرات، الحركات، وشكل الصفحات. اللي هيتغير هو مصدر البيانات وكل اللي ورا الشكل.

٣ كل المشاكل اللي لقيناها

كل مشكلة ليها رقم: **C** = خطيرة (بتوقف الشغل أو ثغرة أمان)، **H** = مهمة، **M** = متوسطة. وجنب كل واحدة المرحلة اللي هتتحل فيها.

🔴 خطيرة (C)

#	المكان	المشكلة	الحل في
C1	`AuthenticationController.CurrentUser`	بياخد الإيميل من العنوان (`?email=`) ويرجّع بيانات صاحبه **وتوكن باسمه**. يعني أي حد مسجل دخول يقدر ياخد حساب أي حد تاني، حتى المدير.	B2 ✓
C2	`appsettings.json`	مفتاح التوكن ومفتاح `Stripe` وكلمة سر المدير (`P@ssw@rd`) مكتوبين في الكود المرفوع.	B0 ✓
C3	`OrderSpecifications`	أي طلب شراء كان بيقع: `Include` على خاصية مش موجودة، وعمود ناقص.	B1 ✓
C4	`ProductsController`	الزائر مكانش يقدر يشوف المنتجات من غير تسجيل دخول، فالمتجر نفسه مقفول.	B1 ✓
C5	`IdentityDataSeeding`	حساب المدير مكانش بيتعمل: مسافات في اسم المستخدم، والدور اللي اتعمل `Admins` والمطلوب `Admin` .	B1 ✓
C6	`Program.cs`	الـ `API` مش بيشتغل أصلاً من غير `Redis` ، وهو مش متسطب ولا موجود على الاستضافة المجانية.	B1 ✓
C7	`ecommerce-web/src`	الفرونت كله وهمي: المنتجات من `dummyjson.com` ، وتسجيل الدخول مزيف وكلمة السر بتتخزن في المتصفح زي ما هي.	F0 / F1

🟠 مهمة (H)

#	المكان	المشكلة	الحل في
H1	`StoreIdentityDbContext`	قاعدتين بيانات منفصلتين، والطلب مربوط بالإيميل كنص بدل ما يكون مربوط بالمستخدم.	B1 ✓
H2	`BasketService`	السعر جاي من المتصفح ومتصدّق، فأي حد يقدر يشتري بأي سعر.	B1 ✓
H3	`DataSeeding`	ملفات البيانات بتتقري بمسار نسبي `.\` فهتفشل على السيرفر.	B1 ✓
H4	`PaymentService`	المبلغ بيتحول لقروش بقص مش تقريب، فبيضيع قروش.	B1 ✓
H5	`OrderService.CreateOrder`	لو السلة من غير دفع، السيرفر بيقع بخطأ ٥٠٠ بدل رسالة واضحة.	B1 ✓
H6	كل المشاريع	`AutoMapper` : نسخة فيها ثغرة أمان، والنسخ الجديدة بقت بفلوس.	B0 ✓
H7	`AuthenticationService`	التسجيل مش بيدي أي دور، والتوكن مفيهوش الدور، ومفيش سياسة للمدير.	B2 ✓

H8	`Login`	مفيش قفل للحساب ولا حد لعدد المحاولات، فتخمين كلمة السر سهل.	B2 ✓
H9	`OrderService` / `Product`	مفيش مخزون خالص، والطلب مش بيخصم حاجة. (العمود اتعمل في B1، والخصم في B4)	B1 ✓ / B4
H10	الـ `API`	مفيش أي روابط للمدير: لا إضافة منتج، ولا رفع صور، ولا إدارة طلبات.	B3 / B4
H11	`PaymentController`	مفيش `webhook` ، فحالة الطلب مش بتتغير لما الدفع ينجح أو يفشل.	B5
H12	`SlideProducts.jsx` , `About.jsx`	استيراد ملفات بحروف كبيرة وصغيرة مختلفة عن اسمها، فالبناء بيقع على لينكس (`Vercel`).	F0

🟡 متوسطة (M)

#	المكان	المشكلة	الحل في
M1	أسماء المشاريع	أخطاء إملائية: `PerisistenceLayer` ، `ApplicationUset` ، `Exteption` .	B0 ✓
M2	`PaginatedResult`	عدد العناصر وحجم الصفحة كانوا بيرجعوا غلط.	B0 ✓
M3	`PictureUrlResolver`	روابط الصور على منفذ غلط ثابت في الكود.	B0 ✓
M4	`GetOrderById`	مكانش بيرجّع "مش موجود" أبداً، وكان بيقع.	B0 ✓
M5	`E-Commerce.Web`	بقايا قالب المشروع (`WeatherForecast` ، `Models/Product.cs`).	B0 ✓
M6	كل المشاريع	تحذيرات كتير، و `nullable` مش مفعّل صح.	B0 ✓
M7	`GetAllOrders`	لو مفيش طلبات بيرجّع ٤٠٤ بدل قايمة فاضية.	B1 ✓
M8	`delivery.json`	أسعار الشحن ١٠ و٥ و٢ (شكلها دولار) والمتجر بالجنيه.	B1 ✓
M9	`emailExists`	رابط مفتوح يقول الإيميل متسجل ولا لأ، من غير أي حد لعدد الطلبات.	B2 ✓
M10	المشروع كله	مفيش ولا اختبار تلقائي.	B7 / F6
M11	`CategoryPage` , `SearchResults`	مفيش فلترة ولا ترتيب ولا ترقيم صفحات في الفرونت.	F2
M12	`CartContext.jsx`	السلة في `localStorage` بس، مش متزامنة مع السيرفر.	F3
M13	`package.json`	إعدادات `gh-pages` ومسار `/Ecommerce-Website/` هتبوظ الروابط على `Vercel` .	F0

الحصيلة: ٣٢ مشكلة · اتحل منهم ٢٣ في B0 و B1 و B2 · الباقي ٩ موزعين على المراحل الجاية (ومنهم H9 اتحل نصها).

٤ القرارات المعمارية والتقنيات

القرار	اخترنا	ليه

الفرونت إند	`React + Vite` ونحوله لـ `TypeScript`	نحافظ على تصميمك، والسيرة الذاتية يبقى فيها رياكت لوحده جنب `Next.js` في مشروع الجيم.
السلة	جدولين في `SQL Server`	`Redis` مش متاح على الاستضافة المجانية، والسلة تبان في الداتا بيز وتترابط بالمستخدم.
قاعدة البيانات	واحدة: `ECommerceStore`	الطلب يترابط بالمستخدم بمفتاح حقيقي، ونشر أسهل (الاستضافة المجانية بتدي قاعدة واحدة).
الكاش	`Output Cache` جوه الـ `API`	صفحات المنتجات بتتحفظ ٥ دقايق من غير سيرفر إضافي، وبتتمسح لما المدير يعدل منتج.
الدفع	`Stripe` تجريبي + كاش عند الاستلام	دفع حقيقي الشكل من غير فلوس، والكاش لأنه الأشهر في مصر.
العملة	جنيه مصري `EGP`	المتجر مصري، والأسعار والشحن كلها بالجنيه.
التحويل بين الكائنات	يدوي (`Mapping extension methods`)	بدل `AutoMapper` : أوضح، أسرع، ومفيش ثغرات ولا رخصة.
الأسرار	`user-secrets` محلياً، ومتغيرات بيئة على السيرفر	مفيش أي سر في الكود المرفوع. مفتاح `Stripe` التجريبي القديم هنسيبه زي ما اتفقنا.
الصلاحيات	`Policies` مش `Roles` مكتوبة في كل حتة	نفس مشروع الجيم: تغيير الصلاحية في مكان واحد.
الاختبارات	`xUnit` + `WebApplicationFactory` على قاعدة حقيقية، و `Playwright` للفرونت	نفس مشروع الجيم، وبيختبر الكود زي ما هيشتغل فعلاً.
النشر	`MonsterASP` للـ `API` و `Vercel` للفرونت	مجانيين، وجربناهم في مشروع الجيم.

الطبقات واعتمادها على بعض

`E-Commerce.Web` → `Presentation` → `ServicesAbstraction` → `Shared`

`Web` → `Services` → `Domain` ← `Persistence`

الـ `Domain` في النص ومش معتمد على حاجة. الـ `Persistence` بقى معتمد على الـ `Domain` بس (بعد ما شلنا الاعتماد على الـ `Services` في B1).

٥ شكل البيانات (الجداول)

قاعدة واحدة اسمها `ECommerceStore` . ده شكلها بعد B1، والبرتقالي جديد:

Users
Id
DisplayName (100)
Email = UserName
PhoneNumber
CreatedAt (idx)

Addresses
Id
UserId → Users (1:1)
FirstName, LastName
City, Street, Country

Roles / UserRoles
Admin
Customer

DeliveryMethods
Id
ShortName
DeliveryTime
Price (EGP)

Products
Id
Name, Description
Price, PictureUrl
Stock ≥ 0
BrandId, TypeId

ProductBrands
Id
Name

ProductTypes
Id
Name

Baskets
Id (string)
UserId? → Users
DeliveryMethodId?
PaymentIntentId, ClientSecret

BasketItems
BasketId → Baskets
ProductId → Products
Quantity 1..100
unique (Basket, Product)

Orders
Id (Guid)
UserId → Users
UserEmail (snapshot)
OrderDate, Status
PaymentMethod
Address (owned)
SubTotal, PaymentIntentId
DeliveryMethodId

OrderItems
Id
OrderId → Orders
ProductId, Name, Picture (snapshot)
Price, Quantity

العلاقات وقواعد المسح

- مستخدم ← طلبات: `Restrict` . مينفعش تمسح عميل عنده طلبات (سجل مالي).
- مستخدم ← سلة / عنوان: `Cascade` . لو الحساب اتمسح، سلته وعنوانه يتمسحوا.
- طلب ← عناصره: `Cascade` .
- عناصر الطلب **مش** مربوطة بالمنتج بمفتاح: نسخة، عشان الطلب القديم يفضل زي ما هو لو المنتج اتعدل أو اتمسح.

قيود جوه القاعدة نفسها

- `CHECK` : السعر > ٠، المخزون ≥ ٠، كمية السلة من ١ لـ ١٠٠.
- منتج واحد مرة واحدة بس في نفس السلة (`unique`).
- الحالة وطريقة الدفع متخزنين كنص (`"Pending"`) مش رقم، فتقراهم من الداتا بيز على طول.
- الفلوس `decimal(18,2)` .

اللي هيتزود على الجداول في المراحل الجاية

المرحلة	التغيير	ليه
B3	`Products.IsActive` (`CreatedAt` +)	المدير يخفي منتج بدل ما يمسحه، وترتيب "الأحدث".

B4	`Products.RowVersion` ، `Orders.ShippedAt / DeliveredAt / CancelledAt`	منع بيع نفس آخر قطعة لاتنين في نفس اللحظة، وتاريخ كل مرحلة في الطلب.
B5	`PaymentEvents` (رقم حدث `Stripe` + وقته)	نفس الحدث لو وصل مرتين مايتنفذش مرتين (`idempotency`).

٦ قواعد العمل لكل جزء

المنتجات

- التصفح والبحث مفتوحين للكل من غير تسجيل.
- الاسم مطلوب (لحد ١٠٠ حرف)، السعر أكبر من صفر، المخزون صفر أو أكتر.
- المنتج اللي مخزونه صفر بيظهر "نفذت الكمية" ومينفعش يتحط في السلة.
- المنتج اللي اتباع قبل كده بيتخفي (`IsActive = false`) بدل ما يتمسح.
- الصورة: `jpg/png/webp` بس، ولحد ٢ ميجا، واسم عشوائي على السيرفر.
- أي تعديل من المدير بيمسح كاش المنتجات فوراً.

السلة

- الزائر عنده سلة برقم محفوظ في المتصفح، من غير حساب.
- المتصفح بيبعت رقم المنتج والكمية بس. السعر والاسم والصورة دايماً من الداتا بيز.
- الكمية من ١ لـ ١٠٠، ومش أكتر من المخزون (بيتراجع تاني وقت الطلب).
- لما يسجل دخول: سلة الزائر تترابط بحسابه. ولو عنده سلة قديمة، الاتنين يتدمجوا.
- السلة بتتمسح بعد ما الطلب يتعمل.

الطلبات

- لازم تسجيل دخول، وعنوان، وطريقة شحن، وسلة مش فاضية.
- الإجمالي = مجموع المنتجات + الشحن، ويتحسب على السيرفر بس.
- المخزون بيتخصم وقت الطلب في `Transaction` واحدة. لو منتج مفيش منه كفاية، الطلب كله يترفض بـ ٤٠٩.
- العميل يلغي طلبه وهو لسه `Pending` بس، والإلغاء يرجّع المخزون.
- العميل يشوف طلباته هو بس، والمدير يشوف الكل.

`Pending` → `PaymentReceived` → `Shipped` → `Delivered`

`Pending` → `PaymentFailed` · `Pending` → `Cancelled` · `COD: Pending` → `Shipped` → `Delivered`

الدفع

- الكارت: السيرفر بيعمل `Payment Intent` بالمبلغ اللي هو حسبه (بالقرش، بالتقريب).
- الطلب بيبقى مدفوع بس لما `Stripe` نفسه يبعت `webhook` ، مش لما المتصفح يقول.
- توقيع الـ `webhook` بيتراجع، والحدث المكرر مابيتنفذش مرتين.
- الكاش عند الاستلام: الطلب بيبدأ `Pending` ، والمدير يعلّمه اتشحن ثم اتسلم.
- كروت التجربة: `4242 4242 4242 4242` (نجاح) و `4000 0000 0000 9995` (رفض).

👤 الحسابات

- الإيميل فريد، وهو نفسه اسم المستخدم.
- كلمة السر: ٨ حروف على الأقل، فيها حرف كبير وصغير ورقم.
- التسجيل العام بيدي دور "عميل" بس. الدور عمره ما بييجي من الفورم.
- ٥ محاولات غلط ← الحساب يتقفل ١٠ دقايق. وحد أقصى ١٠ محاولات دخول في الدقيقة لكل جهاز.
- تغيير كلمة السر لازم معاه القديمة.

📊 الإحصائيات

- للمدير بس.
- المبيعات بتتحسب من الطلبات المدفوعة واللي اتسلمت بس، مش الملغية ولا اللي فشل دفعها.
- "قرب يخلص" = مخزون أقل من ٥ (قيمة في الإعدادات).
- الفترة الافتراضية آخر ٣٠ يوم، وتقدر تختار من / إلى.

٧ الأدوار والصلاحيات

الجزء	زائر	عميل	مدير
تصفح المنتجات والماركات والأنواع	✓	✓	✓
السلة	✓	✓ **(مربوطة بحسابه)**	✓
عمل طلب ودفع	✗	✓	✓
عرض الطلبات	✗	**طلباته بس**	**الكل**
إلغاء طلب	✗	**طلبه وهو Pending**	✓
البروفايل والعنوان وكلمة السر	✗	**بتوعه بس**	**بتوعه بس**
إضافة وتعديل المنتجات والصور والمخزون	✗	✗	✓
الماركات والأنواع	**قراءة**	**قراءة**	**كامل**
تغيير حالة الطلب (شحن / تسليم)	✗	✗	✓
الإحصائيات	✗	✗	✓
قايمة العملاء	✗	✗	✓ **(قراءة)**

هيتطبق إزاي في الكود؟ بـ `Policies` : `[Authorize(Policy = "AdminOnly")]` على كل روابط `/api/admin/*` . و"العميل يشوف بتاعه بس" بيتطبق إن رقم المستخدم بييجي من التوكن نفسه (`sub`)، عمره ما بييجي من العنوان أو الفورم. ده بالظبط اللي هيسد الثغرة C1.

٨ قايمة الـ API Endpoints

الجزء	الروابط	الصلاحية	المرحلة

Products	GET /api/products ? brandId&typeId&sort&search&pageIndex&pageSize GET /api/products/{id} · GET /api/products/brands · GET /api/products/types	عام	B1 ✓
Basket	GET /api/basket?id= · POST /api/basket · DELETE /api/basket/{id}	عام	B1 ✓
Orders	POST /api/order · GET /api/order · GET /api/order/{id} GET /api/order/deliveryMethods	عميل	B1 ✓
Payment	POST /api/payment/{basketId}	عميل	B1 ✓
Auth	POST /api/authentication/login · register (rate limited) GET emailExists (rate limited) · GET currentUser (from token)	عام / عميل	B2 ✓
Account	GET|PUT /api/account/profile · GET|PUT /api/account/address POST /api/account/change-password GET /api/basket/mine · POST /api/basket/{id}/claim	عميل	B2 ✓
Admin Users	POST /api/admin/users/unlock	مدير	B2 ✓
Admin Products	POST /api/admin/products · PUT /{id} · DELETE /{id} POST /{id}/image (multipart) · PATCH /{id}/stock POST|PUT|DELETE /api/admin/brands · /api/admin/types	مدير	B3
+Orders	POST /api/order/{id}/cancel GET /api/admin/orders ?status&from&to&search&page · GET /{id} PATCH /api/admin/orders/{id}/status	عميل / مدير	B4
+Payment	POST /api/payment/webhook (Stripe signature) POST /api/order (paymentMethod: Card | CashOnDelivery)	/ Stripe عميل	B5
Analytics	GET /api/admin/analytics/summary ?from&to · /sales-by-day /best-sellers · /low-stock · /orders-by-status · GET /api/admin/customers	مدير	B6

٩ المميزات حسب الأولوية

Core (لازم)

- تصفح وبحث وفلترة وترتيب وصفحات
- تفاصيل المنتج والمخزون
- سلة متزامنة مع السيرفر
- تسجيل ودخول حقيقي بالتوكن
- شراء بخطوات: عنوان ← شحن ← دفع
- Stripe تجريبي + كاش عند الاستلام
- طلباتي وحالة كل طلب
- لوحة مدير: منتجات وطلبات
- اختبارات تلقائية
- منشور على الإنترنت

Pro (يفرق في الإنترفيو)

- رفع صور المنتجات
- إحصائيات برسوم بيانية
- حماية من التخمين وقفل الحساب
- منع بيع آخر قطعة لاتنين (`RowVersion`)
- Webhook بتأكيد تلقائي
- كاش للمنتجات بيتمسح عند التعديل
- عربي وإنجليزي ووضع ليلي
- صفحة ٤٠٤ وحالات تحميل وأخطاء

Bonus (لو فيه وقت)

- المفضلة محفوظة في الحساب
- تقييمات المنتجات
- كوبونات خصم
- إيميل تأكيد الطلب
- تصدير الطلبات `CSV`
- نسيت كلمة السر بالإيميل

١٠ الجدول الزمني وترتيب المراحل

الحجم: **S** = جلسة واحدة · **M** = ٢ جلسة · **L** = ٣ جلسات تقريباً. الجلسة = يوم شغل مركز.

المرحلة	الاسم	الحجم	معتمدة على	الحالة
B0	الأساس: ترقية، تنضيف، أمان	M	—	خلصت
B1	قاعدة البيانات والبيانات التجريبية	L	B0	خلصت
B2	الحسابات والأمان	M	B1	خلصت
B3	المنتجات وإدارتها	L	B2	الحالية
B4	الطلبات والمخزون	M	B3	لسه
B5	الدفع	M	B4	لسه
B6	الإحصائيات	S	B4	لسه
B7	الاختبارات	L	B2 → B6	لسه
F0	تجهيز الفرونت و TypeScript	M	B3	لسه
F1	الدخول والتسجيل	S	F0 + B2	لسه
F2	المتجر	L	F0	لسه
F3	السلة والشراء وطلباتي	L	F1 + B5	لسه
F4	لوحة المدير	L	F1 + B6	لسه

F5	التشطيب	M	F2 → F4	لسه
F6	الجودة والاختبارات	M	F5	لسه
D1	النشر والريدمي	M	كله	لسه

الإجمالي تقريباً ٢٨–٣٠ جلسة. خلص منهم ٧. الترتيب ده زي مشروع الجيم: الباك إند يتقفل ويتختبر الأول، فالفرونت يتبني فوق حاجة ثابتة ومش هنرجع نغير الروابط.

١١ مراحل الباك إند B0 → B7

B0 الأساس: ترقية وتنضيف وأمان

M · ٢ جلسة ✓ خلصت · 12f8c69

الهدف: مشروع نضيف بيتبني من غير أي تحذير، من غير أسرار في الكود، وجاهز يتكلم مع الفرونت.

المهام

- ✅ تجميع الريبوين في ريبو واحد، والرياكت في فولدر `ecommerce-web`، وفرع `fullstack`.
- ✅ الترقية لـ `.NET 10` وكل المكتبات لآخر نسخة.
- ✅ تصليح الأسماء الغلط (M1) وشيل بقايا القالب (M5).
- ✅ شيل `AutoMapper` وكتابة التحويل يدوي (H6).
- ✅ إخراج الأسرار لـ `user-secrets`، والـ `API` يرفض يشتغل لو مفتاح التوكن مش موجود (C2).
- ✅ `CORS` لعنوان الفرونت، وزرار `Authorize` في `Swagger`.
- ✅ `GlobalExceptionHandler` وأخطاء بشكل `ProblemDetails` موحد.
- ✅ تصليح الترقيم (M2) وروابط الصور (M3) وعرض طلب واحد (M4).
- ✅ `TreatWarningsAsErrors` و `Nullable` لكل المشاريع (M6).

الملفات الأساسية

`Directory.Build.props` `Program.cs` `Extensions/ServiceCollectionExtensions.cs` `Infrastructure/GlobalExceptionHandler.cs` `Infrastructure/RequestPictureUrlBuilder.cs` `Services/Mapping/*.cs`

شروط الانتهاء

- `dotnet build` بصفر تحذيرات.
- مفيش أي سر في أي ملف متتبع في `git`.
- كل خطأ بيرجع `ProblemDetails`.

```
merge: Merge phase B0: foundation, .NET 10, secrets out, zero warnings
```

B1 قاعدة البيانات والبيانات التجريبية

`L` ٣٠ جلسات ✓ خلصت · `dd1c7d7`

الهدف: قاعدة بيانات واحدة واضحة، الـ `API` يشتغل على جهازك من غير `Redis` ، وبيانات واقعية تعرض بيها المشروع.

المهام

- ✅ دمج القاعدتين في `ECommerceStore` و `StoreDbContext : IdentityDbContext` (H1).
- ✅ السلة جدولين `Baskets` و `BasketItems` ، وشيل `Redis` كله (C6).
- ✅ السعر من الكتالوج دايماً، والمتصفح يبعت رقم المنتج والكمية بس (H2).
- ✅ `Product.Stock` ، و `PaymentMethod` ، وحالات طلب كاملة.
- ✅ قيود `CHECK` وقواعد المسح (`Restrict / Cascade`).
- ✅ الطلب مربوط بـ `UserId` من الـ `sub` في التوكن بدل الإيميل.
- ✅ تصليح: الطلب بيقع (C3)، ٥٠٠ من غير دفع (H5)، تقريب المبلغ (H4)، ٤٠٤ للقايمة الفاضية (M7).
- ✅ المنتجات مفتوحة للزائر (C4) مع `Output Cache` ٥ دقايق.
- ✅ ملفات البيانات جوه الـ `dll` (`EmbeddedResource`) (H3)، وأسعار الشحن بالجنيه (M8).
- ✅ حساب المدير من الإعدادات وكلمة سره من `user-secrets` (C5).
- ✅ بيانات تجريبية بأمر `-seed-demo` : ٣٠ عميل، ٨٠ طلب على ٣ شهور، سلة مفتوحة.
- ✅ رسم الجداول (`Mermaid ERD`) والتشغيل المحلي في الريدمي.

الملفات الأساسية

`Persistence/Data/DbContexts/StoreDbContext.cs`

`Persistence/Data/Configurations/*.cs`

`Persistence/Data/Migrations/…_InitialCreate`

`Persistence/Data/DataSeed/DataInitializer.cs`

`Persistence/Data/DataSeed/IdentityDataInitializer.cs`

`Persistence/Data/DataSeed/DemoDataSeeding.cs`

`Persistence/Repositories/BasketRepository.cs`

`Services/OrderService.cs`

`Services/PaymentService.cs`

`Web/Extensions/WebApplicationExtensions.cs`

`README.md`

الأرقام في القاعدة

- ١٣ منتج · ٦ ماركات · ٤ أنواع · ٤ طرق شحن
- ٣٢ مستخدم · ٨٠ طلب · ١٦٦ عنصر

حسابات التجربة

- `customer@demo.store`
- `admin@demo.store`
- كلمة السر: `Demo@2026`

شروط الانتهاء

- القاعدة بتتبني من الصفر بأمر واحد ومن غير أخطاء.
- سكريبت التجربة عدّى كله: الكتالوج للزائر، حماية السعر، الدخول، الطلبات، ٤٠٤ لطلب حد تاني.

```
merge: Merge phase B1: one database, baskets in SQL, seeding + demo data, ERD
```

B2 الحسابات والأمان

M · ٢ جلسة | ✓ خلصت · 9013361

الهدف: كل واحد يدخل بحسابه بأمان، يشوف ويعدل بياناته هو بس، والمدير بس هو اللي يوصل لروابط الإدارة.

المهام

- ✅ سد الثغرة C1: `CurrentUser` بيقرا المستخدم من التوكن، ومابيرجعش توكن خالص.
- ✅ التسجيل: دور `Customer` تلقائي، الإيميل هو اسم المستخدم، إيميل متسجل ← ٤٠٩، كلمة سر ضعيفة ← ٤٠٠ لكل قاعدة.
- ✅ رد الدخول فيه الأدوار ووقت انتهاء التوكن، والتوكن فيه `role` و `jti`.
- ✅ سياسة `AdminOnly`، و ٤٠١ / ٤٠٣ بشكل `ProblemDetails` موحد.
- ✅ `AccountController`: البروفايل، العنوان المحفوظ، تغيير كلمة السر (بالقديمة).
- ✅ السلة ليها صاحب: سلة عميل محدش غيره يقراها أو يعدلها أو يدفع بيها (٤٠٤).
- ✅ ربط سلة الزائر بالحساب بعد الدخول (`claim`) ودمجها مع سلته القديمة، و `basket/mine` لجهاز جديد.
- ✅ قفل الحساب ١٠ دقايق بعد ٥ محاولات غلط، والمدير يقدر يفكه (`admin/users/unlock`).
- ✅ `Rate Limiting`: ١٠ محاولات في الدقيقة لكل جهاز على الدخول والتسجيل و `emailExists` ← ٤٢٩.
- ✅ قواعد كلمة السر، إيميل فريد، مدة التوكن من الإعدادات، و `ClockSkew` دقيقة واحدة.
- ✅ العنوان بيتراجع على أطوال الأعمدة، فالقيمة الطويلة ← ٤٠٠ بدل ٥٠٠.

الملفات الأساسية

`Services/AuthenticationService.cs`
`Services/AccountService.cs` `Services/BasketService.cs`
`Domain/…/CustomerBasket.cs`
`Presentation/Controllers/AccountController.cs`
`Presentation/Controllers/Admin/AdminUsersController.cs`
`Presentation/Controllers/BasketController.cs`
`Presentation/AuthPolicies.cs`
`Web/Extensions/ServiceCollectionExtensions.cs`

الاختبار

- ٤٤ فحص على قاعدة مؤقتة اتمسحت بعدها: كلهم عدّوا.
- الـ ١١ محاولة: أول ١٠ ← ٢٠٠، الحادية عشر ← ٤٢٩.
- اختبار B1 كله اتعاد وعدّى.

شروط الانتهاء

- مفيش أي رابط بياخد رقم مستخدم أو إيميل من برا التوكن.
- عميل على `/api/admin/*` ← ٤٠٣، ومن غير توكن ← ٤٠١.
- المحاولة الخامسة الغلط بتقفل الحساب.
- صفر تحذيرات، والريدمي فيه جدول الروابط بالصلاحيات.

```
merge: Merge phase B2: account takeover fixed, roles and policies, account endpoints, owner-only baskets, lockout and rate limiting
```

B3 المنتجات وإدارتها

L · ٣٠ جلسات | الحالية ⏳

الهدف: المدير يضيف ويعدل ويخفي منتجات بصورها ومخزونها، والزائر يلاقي اللي عايزه بسهولة.

المهام

- [ ] بحث أحسن (في الاسم والوصف)، وفلتر سعر من / إلى، وفلتر "متاح بس"، وترتيب "الأحدث".
- [ ] حد أقصى لحجم الصفحة (٥٠) وقيم افتراضية آمنة.
- [ ] `Products.IsActive` و `CreatedAt` بـ `migration` جديد، والزائر مايشوفش المخفي.
- [ ] `AdminProductsController` : إضافة، تعديل، إخفاء / مسح، تعديل المخزون.
- [ ] رفع صورة (`multipart`): نوع الملف وحجمه بيتراجعوا، واسم عشوائي، والصورة القديمة تتمسح.
- [ ] إدارة الماركات والأنواع (مينفعش تمسح ماركة عليها منتجات ← ٤٠٩).
- [ ] مسح كاش المنتجات (`EvictByTag("catalog")`) بعد أي تعديل.
- [ ] `FluentValidation` أو `DataAnnotations` للمدخلات، وأخطاء ٤٠٠ بالتفاصيل لكل حقل.

الملفات الأساسية

`Presentation/Controllers/Admin/AdminProductsController.cs`
`Presentation/Controllers/Admin/AdminCatalogController.cs` `Services/ProductService.cs`
`Services/ImageStorage.cs` `Shared/ProductQueryParams.cs`
`Services/Specifications/ProductSpecificationHelper.cs`

شروط الانتهاء

- منتج جديد بصورة يظهر في الكتالوج فوراً (الكاش اتمسح).
- صورة `.exe` أو أكبر من ٢ ميجا ← ٤٠٠.
- منتج مخفي مايظهرش للزائر، والطلبات القديمة اللي فيه سليمة.

```
commits: feat(catalog): price/stock filters and newest sort · feat(admin): product CRUD with image upload · feat(admin): brands and types management
```

B4 الطلبات والمخزون

M · ٢٠ جلسة | لسه

الهدف: طلب كامل من أوله لآخره: المخزون يتخصم صح، والحالة تمشي بترتيب منطقي، والمدير يدير الطلبات.

المهام

- [ ] خصم المخزون وقت الطلب في `Transaction` ، ولو مش كفاية ← ٤٠٩ باسم المنتج.
- [ ] `RowVersion` على المنتج، عشان طلبين في نفس اللحظة مايشتروش آخر قطعة مرتين.
- [ ] قواعد تغيير الحالة في مكان واحد (`state machine`)، وأي انتقال غلط ← ٤٠٠.
- [ ] العميل يلغي طلبه وهو `Pending` ، والمخزون يرجع.
- [ ] `AdminOrdersController` : قايمة بفلترة بالحالة والتاريخ والبحث وصفحات، تفاصيل، تغيير الحالة.
- [ ] تواريخ `ShippedAt / DeliveredAt / CancelledAt` بـ `migration` .
- [ ] مراجعة التكرار: نفس السلة ماتعملش طلبين.

الملفات الأساسية

`Services/OrderService.cs` `Domain/Entities/OrderModule/Order.cs`
`Domain/Entities/OrderModule/OrderStatusTransitions.cs`

`Presentation/Controllers/Admin/AdminOrdersController.cs` `Services/Specifications/OrderSpecifications.cs`

شروط الانتهاء

- منتج مخزونه ١ وطلبين في نفس الوقت ← واحد بس ينجح.
- `Delivered → Pending` ← ٤٠٠.
- إلغاء الطلب يرجّع المخزون بالظبط.

```
commits: feat(orders): stock reservation with concurrency token · feat(orders): status transitions and customer cancel · feat(admin): orders management
```

B5 الدفع

لسه · M · ٢٠ جلسة

الهدف: دفع بكارت تجريبي يغيّر حالة الطلب لوحده، ودفع عند الاستلام كبديل.

المهام

- [ ] الطلب بياخد `paymentMethod` : `Card` محتاج `Payment Intent` ، و `CashOnDelivery` لأ.
- [ ] `POST /api/payment/webhook` : التحقق من التوقيع بـ `Stripe:WebhookSecret` .
- [ ] `payment_intent.succeeded` ← `PaymentReceived` ، و `payment_failed` ← `PaymentFailed` .
- [ ] جدول `PaymentEvents` عشان الحدث المكرر مايتنفذش مرتين.
- [ ] لو مبلغ السلة اتغير بعد الـ `Intent` ، يتحدث قبل الدفع.
- [ ] التجربة محلياً بـ `Stripe CLI` (`stripe listen --forward-to`).

الملفات الأساسية

`Services/PaymentService.cs` `Presentation/Controllers/PaymentController.cs`
`Domain/Entities/OrderModule/PaymentEvent.cs` `Services/OrderService.cs`

شروط الانتهاء

- كارت `4242…` ← الطلب يبقى `PaymentReceived` من غير أي تدخل.
- `webhook` بتوقيع غلط ← ٤٠٠ ومفيش تغيير.
- طلب كاش يتعمل من غير `Stripe` خالص.

```
commits: feat(payment): cash on delivery option · feat(payment): stripe webhook with signature check and idempotency
```

B6 الإحصائيات

لسه · S · جلسة

الهدف: أرقام جاهزة للوحة المدير، محسوبة في الداتا بيز مش في الذاكرة.

المهام

- [ ] ملخص: إجمالي المبيعات، عدد الطلبات، متوسط الطلب، عملاء جداد، في فترة من / إلى.
- [ ] المبيعات لكل يوم (للرسم البياني)، والطلبات حسب الحالة.
- [ ] الأكثر مبيعاً (بالكمية والفلوس)، والمنتجات اللي قربت تخلص.
- [ ] قايمة العملاء للمدير (بحث وصفحات، وعدد طلبات كل عميل).
- [ ] استعلامات بـ `GroupBy` و `AsNoTracking` ، وفهرس على `OrderDate` و `Status` .

الملفات الأساسية

`Services/AnalyticsService.cs` `Presentation/Controllers/Admin/AdminAnalyticsController.cs`
`Shared/DTOs/AnalyticsDTOs/*.cs`

شروط الانتهاء

- الأرقام مطابقة لاستعلام `SQL` يدوي على البيانات التجريبية.
- الطلبات الملغية والفاشلة مش داخلة في المبيعات.

```
commit: feat(admin): sales analytics, best sellers, low stock
```

B7 الاختبارات

`L ٣٠ جلسات` لسه

الهدف: ثقة إن أي تعديل بعد كده مش هيبوظ حاجة، زي مشروع الجيم.

المهام

- [ ] مشروع `E-commerce.Tests` بـ `xUnit` و `FluentAssertions`.
- [ ] `Unit tests` : حساب الإجمالي، تقريب المبلغ، دمج السلة، انتقالات الحالة.
- [ ] `Integration tests` بـ `WebApplicationFactory` على قاعدة `SQL Server` مؤقتة بتتمسح بعدها.
- [ ] اختبار صلاحيات لكل رابط: زائر / عميل / مدير ← الكود المتوقع (٢٠٠ / ٤٠١ / ٤٠٣).
- [ ] سيناريو كامل: تسجيل ← سلة ← طلب ← `webhook` ← المدير يشحن ← يتسلم.
- [ ] اختبار التزامن على آخر قطعة، واختبار الثغرة C1 عشان ماترجعش.
- [ ] `GitHub Actions` : بناء واختبارات على كل `push` .

الملفات الأساسية

`tests/E-commerce.Tests/*` `tests/…/ApiFactory.cs` `tests/…/AuthorizationMatrixTests.cs`
`.github/workflows/ci.yml`

شروط الانتهاء

- كل الاختبارات خضرا محلياً وعلى `GitHub Actions` .
- كل رابط في القسم ٨ ليه اختبار صلاحية.

```
commits: test: unit tests for pricing and order rules · test: integration tests with real database
· ci: build and test workflow
```

١٢ معمارية الفرونت إند

شكل الفولدرات بعد التحويل

```
ecommerce-web/src/
  api/          client.ts (fetch + token + errors)
                products.ts · basket.ts · orders.ts · auth.ts · admin.ts
  types/        Product · Basket · Order · User (from API DTOs)
  context/      AuthContext · BasketContext · ThemeContext
  hooks/        useProducts · useDebounce · useQueryParams
  components/   Header · ProductCard · Pagination · Filters · Spinner …
  pages/        Home · Shop · ProductDetails · Cart · Checkout
                Orders · OrderDetails · Account · Login · Register
                admin/  Dashboard · Products · Orders · Catalog
```

القرارات

- **TypeScript strict**: أنواع مطابقة للـ `DTOs` بتاعة الـ `API` .
- **طبقة `api/` واحدة**: عنوان الـ `API` من `VITE_API_URL` ، والتوكن بيتضاف لوحده، و ٤٠١ يطلعك لصفحة الدخول.

```
i18n/          ar.json · en.json
styles/        (your existing CSS files, kept)
```

- **الحالة**: `Context` للدخول والسلة. مفيش داعي لـ `Redux` في مشروع بالحجم ده.
- **الفلاتر في الرابط** (`?brand=&sort=`) عشان الصفحة تتشارك وزرار الرجوع يشتغل.
- **الحماية**: `ProtectedRoute` للعميل و `AdminRoute` للمدير، والحماية الحقيقية على السيرفر.
- **الدفع**: `@stripe/react-stripe-js` (`Payment Element`).
- **التصميم**: نفس الـ `CSS` بتاعك، ونزود متغيرات للوضع الليلي.

اللي هيتشال

- كل طلبات `dummyjson.com` ، وتسجيل الدخول المزيف وكلمة السر في `localStorage` .
- إعدادات `gh-pages` و `homepage` .
- صفحات المدونة الوهمية لو مالهاش لازمة (قرار ليك في القسم ١٧).

١٣ مراحل الفرونت إند F0 → F6

F0 التجهيز و TypeScript

`M` ٢٠ جلسة — لسه

الهدف: الموقع بيجيب منتجاتك الحقيقية من الـ `API` ، ومكتوب `TypeScript` ، وبيتبني على لينكس.

- ☐ إضافة `TypeScript` و `tsconfig` strict، وتحويل الملفات `.jsx → .tsx` تدريجياً.
- ☐ تصليح أسماء الملفات (H12)، وشيل `gh-pages` (M13).
- ☐ طبقة `api/client.ts` و `.env.example` فيه `VITE_API_URL` .
- ☐ أنواع `types/` من الـ `DTOs` ، وتوصيل الرئيسية والسلايدرات بالمنتجات الحقيقية.
- ☐ `ESLint` و `Prettier` بقواعد `TypeScript` .

`ecommerce-web/tsconfig.json` `src/api/client.ts` `src/types/*.ts` `vite.config.ts`

- ◆ `npm run build` و `npm run lint` من غير أخطاء.
- ◆ مفيش أي ذكر لـ `dummyjson` .

```
commit: chore(web): typescript, api client, real products
```

F1 الدخول والتسجيل

S · جلسة ١ | لسه

الهدف: حسابات حقيقية بالتوكن بدل المزيفة.

- [ ] `AuthContext` : دخول، تسجيل، خروج، والمستخدم الحالي من `currentUser` .
- [ ] رسايل الخطأ من الـ `API` تظهر جنب كل حقل.
- [ ] `ProtectedRoute` و `AdminRoute` ، وقايمة الهيدر تتغير حسب الدور.
- [ ] صفحة الحساب: البروفايل، العنوان، تغيير كلمة السر.
- [ ] ربط السلة بالحساب بعد الدخول.

- مفيش كلمة سر في المتصفح خالص.
- انتهاء التوكن ← رجوع لصفحة الدخول برسالة.

```
commit: feat(web): real authentication and account page
```

F2 المتجر

L · ٣ جلسات | لسه

الهدف: تجربة تسوق كاملة: تلاقي المنتج بسرعة وتشوف تفاصيله.

- [ ] صفحة `Shop` : فلاتر (ماركة، نوع، سعر، متاح)، ترتيب، صفحات، والكل في الرابط.
- [ ] بحث في الهيدر مع `debounce` .
- [ ] تفاصيل المنتج: الصورة، المخزون، "نفذت الكمية"، اختيار الكمية، منتجات مشابهة.
- [ ] حالات التحميل (`skeleton`) والنتايج الفاضية والأخطاء.

- رابط بفلاتر لو اتفتح في تاب جديد يرجع نفس النتايج.

```
commit: feat(web): shop page with filters, search and product details
```

F3 السلة والشراء وطلباتي

L · ٣ جلسات | لسه

الهدف: تشتري من أول المنتج لحد الدفع، وتتابع طلبك.

- [ ] `BasketContext` متزامن مع السيرفر (M12)، ورقم السلة في المتصفح.
- [ ] صفحة شراء بخطوات: العنوان (متملي من الحساب) ← الشحن ← المراجعة ← الدفع.
- [ ] `Stripe Payment Element` للكارت، واختيار الكاش عند الاستلام.
- [ ] صفحة نجاح الطلب، و"طلباتي" بالحالة، وتفاصيل الطلب، وزرار إلغاء.

- شراء كامل بكارت `4242…` والطلب يظهر مدفوع في "طلباتي".

```
commit: feat(web): synced basket, stepped checkout with stripe, my orders
```

F4 لوحة المدير

لسه · L ٣٠ جلسات

الهدف: لوحة تحكم كاملة للمتجر.

- [ ] الرئيسية: كروت الأرقام، رسم المبيعات، الأكثر مبيعاً، قرب يخلص.
- [ ] المنتجات: جدول ببحث وصفحات، فورم إضافة / تعديل برفع صورة، تعديل المخزون.
- [ ] الماركات والأنواع.
- [ ] الطلبات: فلترة بالحالة والتاريخ، التفاصيل، تغيير الحالة.
- [ ] العملاء: قايمة وعدد طلبات كل عميل.

- المدير يضيف منتج بصورته ويلاقيه في المتجر فوراً.

```
commit: feat(web): admin dashboard, products, orders, analytics
```

F5 التشطيب

لسه · M ٢٠ جلسة

الهدف: موقع متشطب زي مشروع الجيم.

- [ ] عربي وإنجليزي مع `RTL` ، ووضع ليلي، والاختيار بيتحفظ.
- [ ] مراجعة الموبايل لكل صفحة.
- [ ] صفحة ٤٠٤، وعناوين الصفحات (`title`)، وأيقونة الموقع.
- [ ] الصفحات الوهمية (المدونة / تواصل) حسب قرارك.

```
commit: feat(web): i18n, dark mode, mobile polish, 404
```

F6 الجودة والاختبارات

لسه · M ٢٠ جلسة

الهدف: ثقة إن مفيش حاجة بايظة، وسرعة كويسة.

- [ ] `Playwright` : زائر يتصفح، عميل يشتري، مدير يضيف منتج.
- [ ] `Vitest` لشوية دوال وكومبوننتس مهمة.
- [ ] `Lighthouse` : تحميل الصفحات تدريجياً (`lazy routes`) وضغط الصور.
- [ ] إضافة اختبارات الفرونت لـ `GitHub Actions` .

```
commit: test(web): e2e flows with playwright, performance pass
```

١٤ النشر والتوثيق D1

D1 النشر والريدمي

لسه · M ٢٠ جلسة

الهدف: لينك حقيقي شغال تحطه في السيرة الذاتية ولينكد إن، بنفس طريقة مشروع الجيم.

- [ ] الـ `API` على `MonsterASP` : `dotnet publish` ، قاعدة `SQL Server` هناك، الأسرار متغيرات بيئة.

- [ ] تشغيل الـ `migrations` والبيانات التجريبية مرة واحدة على السيرفر.
- [ ] الصور المرفوعة في فولدر ثابت على السيرفر.
- [ ] `Stripe webhook` على عنوان السيرفر الحقيقي.
- [ ] الفرونت على `Vercel` : `VITE_API_URL` ، و `vercel.json` للروابط، و `CORS` للعنوان الجديد.
- [ ] الريدمي: صور، رسم المعمارية، الجداول، حسابات التجربة، لينك الموقع، وطريقة التشغيل.
- [ ] تجربة كاملة على الموقع الحقيقي من الموبايل.
- [ ] رفع الكود على `GitHub` (بموافقتك) وبوست لينكد إن.
- ◆ الموقع شغال من أي جهاز، وشراء تجريبي كامل نجح.
- ◆ مفيش أي سر في الريبو.

```
commit: docs: deployment guide, screenshots, live links
```

١٥ طريقة الشغل و Git

Git

- فرع لكل مرحلة: `phase/b2-auth` ، `phase/b3-catalog` ...
- رسائل `Conventional Commits` : `feat` / `fix` / `docs` / `test` .
- دمج في `fullstack` بـ `--no-ff` بعد ما المرحلة تشتغل وتتختبر.
- مفيش رفع على `GitHub` غير بموافقتك.

الأمان والبيانات

- مفيش أسرار في الكود، ولا فولدر `docs/` (محلي بس).
- نسخة احتياطية قبل أي تعديل خطير على الداتا بيز.
- التجارب الخطيرة على قواعد مؤقتة وتتمسح.
- القواعد القديمة (`ECommerce` ، `ECommerce.Identity`) محدش بيلمسها.

الكود

- تعليقات بسيطة بتشرح **ليه** مش إيه.
- صفر تحذيرات (البناء بيقع لو فيه تحذير).
- بيانات واقعية مش كلام عشوائي.

بعد كل مرحلة

- تقرير بالعربي: اتعمل إيه وتجربه إزاي.
- الملف ده بيتحدث، وملف `INTERVIEW_NOTES.md` بيتزود فيه أسئلة المرحلة.
- أي قرار مهم بنتفق عليه الأول.

أوامر هتحتاجها

الأمر	بيعمل إيه
`dotnet run --project E-Commerce.Web`	يشغل الـ `API` (ويبني القاعدة لو مش موجودة)
`dotnet run --project E-Commerce.Web --seed-demo`	يحط البيانات التجريبية (العملاء والطلبات)
`http://localhost:5163/swagger`	صفحة تجربة الروابط
`cd ecommerce-web; npm install; npm run dev`	يشغل الفرونت

`dotnet user-secrets list --project E-Commerce.Web`	يعرض الأسرار المحفوظة على جهازك

١٦ إزاي يتكتب في السيرة الذاتية

ShopSphere: Full-Stack E-commerce Platform

ASP.NET Core 10 · EF Core · SQL Server · React 19 · TypeScript · Stripe · xUnit · Playwright

- Merged a standalone Web API and a React storefront into one full-stack product with a single SQL Server database and Onion architecture.
- Fixed a critical account-takeover flaw and moved all secrets out of source control; added roles, policies, lockout and rate limiting.
- Replaced Redis with SQL-backed baskets and output caching, so the app runs on free hosting with server-side pricing only.
- Built checkout with Stripe (test mode, signed webhooks) and cash on delivery, with stock reservation protected by concurrency tokens.
- Built an admin dashboard: product management with image upload, order status workflow and sales analytics.
- Covered every endpoint with authorization integration tests on a real database, plus end-to-end browser tests in CI.

الاسم `ShopSphere` اقتراح بس، وتقدر تغيره (القسم ١٧). الأرقام (عدد الاختبارات مثلاً) هتتحط بعد B7.

١٧ قرارات محتاجة رأيك

#	القرار	اقتراحي	إمتى
١	اسم المتجر اللي يظهر في الموقع والريدمي	`ShopSphere` ، أو أي اسم تختاره	قبل F0
٢	صفحات المدونة الوهمية	نشيلها، أو نسيبها ثابتة من غير ما تبان كأنها حقيقية	F5
٣	صفحة تواصل معنا (`EmailJS`)	نسيبها لو المفتاح بتاعك شغال	F5
٤	المفضلة	في المتصفح بس دلوقتي، وفي الحساب لو فيه وقت (`Bonus`)	F3
٥	صور المنتجات	نفس صورك الحالية، والمدير يقدر يغيرها	B3
٦	نطاق (`domain`) خاص	نستخدم عناوين `Vercel` و `MonsterASP` المجانية	D1

اتفقنا عليه خلاص: رياكت + `TypeScript` · السلة في `SQL` · قاعدة واحدة · `Stripe` تجريبي + كاش · مفتاح `Stripe` القديم يفضل زي ما هو.

١٨ قاموس المصطلحات

المصطلح	معناه ببساطة
`API`	الباك إند: البرنامج اللي على السيرفر، بيستقبل الطلبات من الموقع ويرد بالبيانات.
`Full Stack`	مشروع فيه الواجهة والباك إند وقاعدة البيانات شغالين مع بعض.

Onion Architecture	تقسيم الكود لطبقات، والقلب (Domain) مش معتمد على حاجة.
Migration	ملف بيوصف شكل الجداول، والبرنامج بيبني القاعدة منه تلقائي.
Seed	البيانات اللي بتتحط في القاعدة أول ما تتعمل.
JWT / توكن	"كارنيه" رقمي بياخده المستخدم بعد الدخول، وبيبعته مع كل طلب.
Claim	معلومة جوه التوكن، زي رقم المستخدم أو دوره.
Policy	قاعدة صلاحية بإسم، زي "مدير بس"، بتتحط على الرابط.
٤٠١ / ٤٠٣	٤٠١ = مش مسجل دخول. ٤٠٣ = مسجل بس مش مسموحلك.
Rate Limiting	حد أقصى لعدد الطلبات في الدقيقة، عشان محدش يجرب آلاف كلمات السر.
CORS	إذن من الـ API لموقع على عنوان تاني إنه يكلمه.
Output Cache	السيرفر بيحتفظ بالرد شوية، فالطلب المكرر بيترد فوراً من غير الداتا بيز.
Transaction	مجموعة عمليات يا تنجح كلها يا تتلغي كلها.
RowVersion	رقم بيتغير مع كل تعديل للصف، فلو اتنين عدلوا في نفس اللحظة التاني يترفض.
Webhook	رسالة Stripe بيبعتها للسيرفر بنفسه لما الدفع ينجح أو يفشل.
Payment Intent	"نية دفع" بمبلغ محدد بيعملها السيرفر عند Stripe قبل ما العميل يدخل الكارت.
Snapshot	نسخة من البيانات وقت الطلب، عشان التعديلات اللي بعد كده ماتأثرش عليه.
user-secrets	مكان على جهازك بس للأسرار، بعيد عن الكود المرفوع.
TypeScript	جافاسكريبت بأنواع: بيكشف أخطاء كتير قبل ما الموقع يشتغل.
CI	بناء واختبار تلقائي على GitHub مع كل رفع كود.
فرع (branch)	نسخة منفصلة من الكود بنشتغل عليها، وبعدين ندمجها.